Java 8 Interview Preparation Notes

A compact, searchable reference PDF for practicing LangChain PDF loading, page-aware chunking, FAISS retrieval, and grounded interview answers.

Contents: Functional Interfaces and Lambdas; Stream API; Optional; Default Methods; Date and Time API; CompletableFuture; Interview Practice.

How to use: Save this PDF under `data/pdfs/java_interview_notes.pdf`, run the PDF RAG indexer, and ask questions about these topics. The page references shown by your retriever should correspond to the topic pages below.

Scope: Educational notes on Java 8 features, not a comprehensive Java language specification.

1. Functional Interfaces and Lambda Expressions

Functional interfaces

A functional interface has exactly one abstract method. It can have default and static methods without losing functional-interface status. The @FunctionalInterface annotation is optional but helps the compiler catch accidental violations. Common examples include Runnable, Callable, Predicate, Function, Consumer, and Supplier.

Interview checks: What is a functional interface? Can it contain default methods?

Lambda expressions

A lambda expression provides a concise implementation of a functional interface. Lambda syntax consists of parameters, an arrow, and an expression or block body. Lambdas can capture local variables only when those variables are final or effectively final.

```
Predicate<String> nonEmpty = s -> !s.isEmpty();
System.out.println(nonEmpty.test("Java"));
```

Interview checks: How are lambdas different from anonymous classes? What is effectively final?

Method references

Method references are compact forms of lambdas that invoke an existing method. Common forms include ClassName::staticMethod, instance::method, ClassName::instanceMethod, and ClassName::new.

```
List<String> names = List.of("Ada", "Lin");
names.forEach(System.out::println);
```

Interview checks: When should you prefer a method reference?

2. Stream API and Collectors

Stream processing

The Stream API processes sequences of elements declaratively. A stream pipeline has a source, zero or more intermediate operations, and a terminal operation. Intermediate operations such as filter, map, distinct, and sorted are lazy: they execute when a terminal operation requests results.

Interview checks: Why are intermediate stream operations lazy?

Common operations

filter retains elements matching a predicate; map transforms elements; flatMap flattens nested streams; collect accumulates results; reduce combines values; count, anyMatch, and findFirst are terminal operations. A stream should not be reused after a terminal operation.

```
List<Integer> nums = Arrays.asList(1, 2, 3, 4);
List<Integer> squares = nums.stream()
    .filter(n -> n % 2 == 0)
    .map(n -> n * n)
    .collect(Collectors.toList());
```

Interview checks: What is the difference between map and flatMap?

Parallel streams

parallelStream may process work concurrently using the common ForkJoinPool. Parallel execution is not automatically faster: task size, splitting overhead, ordering, thread safety, and shared mutable state matter.

Interview checks: When should parallel streams be avoided?

3. Optional and Null Handling

Purpose of Optional

Optional represents a value that may or may not be present. It encourages callers to handle absence explicitly and is commonly used as a return type. Optional does not prevent all NullPointerExceptions, and indiscriminate use as fields or parameters is usually discouraged.

Interview checks: Why use Optional instead of returning null?

Creating and consuming Optional

Optional.of(value) requires a non-null value; ofNullable(value) accepts null; empty() creates an absent value. Prefer map, flatMap, filter, ifPresent, orElse, orElseGet, and orElseThrow to unchecked get().

```
Optional<String> name = Optional.ofNullable(input);
String label = name.map(String::trim)
    .filter(s -> !s.isEmpty())
    .orElse("Unknown");
```

Interview checks: What is the difference between orElse and orElseGet?

Eager vs lazy fallback

orElse evaluates its fallback argument eagerly, while orElseGet invokes its Supplier only when the Optional is empty. This distinction matters if the fallback is expensive or has side effects.

Interview checks: When does orElseGet save work?

4. Default Methods and Java Time API

Default methods in interfaces

Java 8 allows interfaces to declare default methods with implementations. They support interface evolution while preserving compatibility for many implementers. A class method takes precedence over an interface default method; conflicts between unrelated interface defaults may require an explicit override.

```
interface Greeter {
    default String greet() { return "Hello"; }
}
```

Interview checks: How are conflicting default methods resolved?

Static interface methods

Interfaces can declare static methods. They are invoked using the interface name and are not inherited as instance methods.

Interview checks: How do static interface methods differ from default methods?

Modern date and time

The java.time package introduced immutable, thread-safe date/time types such as LocalDate, LocalTime, LocalDateTime, Instant, ZonedDateTime, Duration, and Period. Instant represents a point on the UTC timeline, while LocalDateTime has no timezone or offset.

```
LocalDate today = LocalDate.now();
LocalDate nextWeek = today.plusWeeks(1);
```

Interview checks: When should you use Instant versus LocalDateTime?

5. CompletableFuture and Interview Practice

Asynchronous programming

CompletableFuture supports asynchronous computations and composition. supplyAsync produces a result; runAsync performs work without a result; thenApply transforms a result; thenCompose chains dependent asynchronous stages; thenCombine combines independent stages. exceptionally and handle can process failures.

```
CompletableFuture<Integer> future =
    CompletableFuture.supplyAsync(() -> 21)
        .thenApply(n -> n * 2);
```

Interview checks: What is the difference between thenApply and thenCompose?

Threading considerations

Async stages without an explicit Executor may use the common ForkJoinPool. Blocking calls and CPU-heavy tasks can need a dedicated executor. A CompletableFuture is not a guarantee that every operation runs on a new thread.

Interview checks: Why pass a custom Executor to supplyAsync?

Practice scenarios

Scenario A: Filter a list of employee names, convert them to uppercase, and collect the result. Scenario B: Safely extract a possibly missing customer email using Optional. Scenario C: Explain how a class resolves two inherited default methods with the same signature. Scenario D: Describe when parallel streams might perform worse than sequential streams.

Interview checks: Explain the design tradeoffs, not just the syntax.

Out-of-scope question for RAG testing

This document does not provide salary information, Kubernetes deployment instructions, or company-specific hiring policies. A document-grounded assistant should say it lacks sufficient source information for those questions.

Interview checks: What should the assistant do when the PDF does not answer the question?